Data Structures (ETCCDS202)

Unit–2 Assignment: Linear Data Structures

Course Details

Name of the School:	School of Engineering & Technology
Program (Class / Semester / Batch):	B. Tech CSE (AI & ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Number:	2
Unit Title:	Linear Data Structures
No. of Hours:	10
Student Name:	**Shivansh Dahiya**
Roll No.:	**2501730145**

Section: A

Submission Date: 17 April,2026

1. Introduction

We used different kinds of Linear Data Structures for this assignment, like Dynamic Arrays, Linked Lists, Stack, and Queue. These structures keep data in order.

We also made a real-world program with Stack to see if the parentheses in an expression are balanced.

2. Dynamic Array Implementation

A Dynamic Array is an array that automatically increases its size when it becomes full.

Working

- Initially, the array starts with a fixed capacity.
- When the array becomes full:
  - A new array of double size is created.
  - All elements are copied into the new array.

- Then the new element is added.

2.1 Time Complexity Analysis

- Most append operations take **O(1)** time.
- When resizing occurs, it takes **O(n)** time because elements are copied.
- Resizing happens rarely.

👉 Therefore, the **amortized time complexity of append() is O(1).**

3. Singly Linked List

A Singly Linked List consists of nodes where each node contains:

- Data
- Pointer to the next node

Operations Implemented

- Insert at beginning → O(1)
- Insert at end → O(n)
- Delete by value → O(n)
- Traverse → O(n)

Advantages

- Dynamic size
- No shifting required like arrays

4. Doubly Linked List

A Doubly Linked List contains:

- Data
- Pointer to previous node
- Pointer to next node

Operations Implemented

- Insert after a node
- Delete at a specific position

Advantages

- Traversal in both directions
- Easier deletion compared to singly linked list

5. Stack Implementation (Using Linked List)

Stack follows **LIFO (Last In First Out)** principle.

Operations

- Push → Insert element at top
- Pop → Remove top element
- Peek → View top element

Time Complexity

- Push → O(1)
- Pop → O(1)
- Peek → O(1)

6. Queue Implementation (Using Linked List)

Queue follows **FIFO (First In First Out)** principle.

Operations

- Enqueue → Add element at rear
- Dequeue → Remove element from front
- Front → View front element

Time Complexity

- Enqueue → O(1)
- Dequeue → O(1)
- Front → O(1)

7. Application: Balanced Parentheses Checker

We implemented a function to check whether an expression has balanced parentheses using Stack.

Logic

- Push opening brackets → ({ [
- When a closing bracket appears:
    - Check if it matches the top of stack
- If mismatch or stack not empty → Not Balanced

7.1 Test Cases

Input	Output
([])	Balanced
([)]	Not Balanced
(((	Not Balanced
""	Balanced

8. Conclusion

In this assignment, we successfully:

- Implemented Dynamic Arrays with resizing logic
- Built Singly and Doubly Linked Lists
- Created Stack and Queue using Linked List
- Solved a real-world problem using Stack

This helped in understanding how data structures work internally and their importance in programming.